

- Informe Técnico: Modelado, Evaluación y Despliegue - Clasificación (Fases 4-6 CRISP-DM)
 - 1. Fase: Modelamiento (Modeling)
 - 1.1. Estrategia de Validación y Mitigación de Data Leakage
 - 1.2. Selección de Variables Predictoras y Target
 - 1.3. Auditoría del Dataset y Decisión de Ingeniería de Cambio de Target
 - 1.4. Selección de Algoritmos
 - 2. Fase: Evaluación (Evaluation)
 - 2.1. Resultados del Rendimiento por Modelo (Test Set)
 - 2.2. Análisis Crítico y Evidencia de la Caída de KNN
 - 2.3. Validación Cruzada (Cross-Validation) para Auditar el Sobreajuste
 - 3. Fase: Despliegue (Deployment)
 - 4. Declaración de Uso de IA Generativa

Informe Técnico: Modelado, Evaluación y Despliegue - Clasificación (Fases 4-6 CRISP-DM)

Autor: Héctor Aguila **Asignatura:** Fundamentos de Machine Learning **Metodología:** CRISP-DM

1. Fase: Modelamiento (Modeling)

1.1. Estrategia de Validación y Mitigación de Data Leakage

Para garantizar la validez científica del experimento y prevenir activamente el **Data Leakage** (fuga de datos) que infla artificialmente el rendimiento, se adoptaron las siguientes directrices estructurales:

- **División Temprana (Train/Test Split):** La separación de los datos en 80% para entrenamiento (2233 registros) y 20% para prueba (559 registros) se realizó **antes**

de aplicar cualquier transformación. Esto evita que los estadísticos globales del dataset (media y desviación estándar) se filtren en la fase de ajuste del modelo.

- **Semilla y Estratificación:** Se fijó una semilla aleatoria (`random_state=42`) para garantizar la reproducibilidad y se estratificó la partición para preservar el balance original de clases (Jugadores MVP y No MVP).
- **Encapsulamiento en Pipelines:** El preprocesamiento de datos (estandarización con `StandardScaler` de numéricas y codificación con `OneHotEncoder` de categóricas) se integró en `Pipelines` independientes para cada algoritmo. Esto asegura que la estandarización aprenda los parámetros (`fit`) únicamente del conjunto de entrenamiento y los aplique de forma aislada al test set.

1.2. Selección de Variables Predictoras y Target

- **Target Categórico:** `mvp_award` mapeado a binario (1 para MVP/Yes, 0 para No MVP/No).
- **Variables Predictoras (Features):** Se descartaron identificadores sin valor predictivo (`record_id`, `player_id`). Se conservaron variables físicas y estadísticas, e incorporamos `match_outcome` (Win/Loss) como variable predictora categórica (ya que al final de la partida el resultado sí influye de manera natural en la asignación del MVP).
- **Traspaso de Target a Feature:** La variable `performance_score` (que en la entrega anterior funcionaba como target de regresión) se incorporó en esta fase como una variable predictora clave.

1.3. Auditoría del Dataset y Decisión de Ingeniería de Cambio de Target

Durante la fase de *Data Understanding*, se auditó la distribución de la variable objetivo inicial `match_outcome` (Victoria/Derrota), revelando un desbalanceo extremo: **2793 victorias vs. solo 7 derrotas**. Con un desbalanceo del 99.75%, cualquier clasificador predice siempre la clase mayoritaria "Win" y obtiene una exactitud perfecta sin aprender nada, lo que invalida el experimento.

Por consiguiente, se tomó la decisión metodológica de usar `mvp_award` como target de clasificación. Cuenta con una distribución realista de **2561 registros "No" vs. 239**

registros "Yes" (un balance aproximado del 90%/10%), lo que expone a los modelos a una clasificación binaria real donde el rendimiento y las distancias se pueden evaluar empíricamente.

1.4. Selección de Algoritmos

Se entrenaron tres algoritmos de clasificación de distintas características y geometrías:

- 1. **Regresión Logística:** Clasificador lineal clásico probabilístico.
- 2. **K-Nearest Neighbors (KNN):** Algoritmo no paramétrico basado en distancias espaciales locales.
- 3. **Random Forest Classifier:** Ensamble no lineal (Bagging) de árboles de decisión no correlacionados.

2. Fase: Evaluación (Evaluation)

2.1. Resultados del Rendimiento por Modelo (Test Set)

El rendimiento de los clasificadores en el conjunto de prueba para predecir la clase positiva **MVP (1)** se detalla a continuación:

Modelo	Accuracy	Precision (MVP)	Recall (MVP)	F1-Score (MVP)	ROC AUC
Regresión Logística	0.9946	0.9592	0.9792	0.9691	0.9980
K-Nearest Neighbors (K=5)	0.9517	0.8621	0.5208	0.6494	0.9995
Random Forest Classifier	1.0000	1.0000	1.0000	1.0000	1.0000

2.2. Análisis Crítico y Evidencia de la Caída de KNN

- **La Debilidad Empírica de KNN:** En las matrices de confusión y reportes del notebook se evidencia la caída en el rendimiento de KNN. Su exactitud global baja a 95.17%, pero lo más crítico es el **F1-Score para la clase MVP, que cae a 0.6494** con un recall de solo 52.08% (dejando de identificar casi a la mitad de los jugadores MVP reales). Esto demuestra geométricamente la sensibilidad de KNN al desbalance de clases y a la dimensionalidad en el espacio de características.
- **Separación Perfecta de Random Forest y Regresión Logística:** Ambos modelos alcanzan un ajuste perfecto o casi perfecto sobre el conjunto de test. Este comportamiento responde a que la variable target **mvp_award** se define mediante una regla sintética determinista perfecta (sin ruido aleatorio, $\epsilon = 0$) basada en si el jugador tiene un alto rendimiento y si el equipo ganó. Tanto las rectas de la regresión logística como los cortes ortogonales de los árboles replican con exactitud la frontera matemática del dataset.

2.3. Validación Cruzada (Cross-Validation) para Auditar el Sobreajuste

Siguiendo las directrices académicas de cuestionar rendimientos perfectos del 100% en el test set para descartar sobreajuste (*overfitting*), se aplicó **Validación Cruzada de 5 pliegues (5-Fold Cross-Validation)** sobre el conjunto de entrenamiento utilizando el F1-Score como métrica:

- **F1-Scores por pliegue:** [1.0000, 0.9867, 1.0000, 1.0000, 0.9867]
- **F1-Score promedio:** 0.9947
- **Conclusión de Robustez:** Las pequeñas fluctuaciones según el pliegue confirman empíricamente que el Random Forest es estable, no memoriza ciegamente la muestra y posee una excelente capacidad de generalización ante datos no vistos.

Selección del Modelo Campeón: Se selecciona **Random Forest Classifier** por su excelente rendimiento general en validación cruzada y test, exportando su pipeline a producción.

3. Fase: Despliegue (Deployment)

- **Pipeline Serializado:** Se importó el pipeline del clasificador desde el archivo serializado [`best_model.joblib`] (`file:///Users/user/Desktop/FundamentosML/03_Modelos_Clasificacion/checkpoints/best_model.joblib`), garantizando que las transformaciones se apliquen de forma transparente en producción.
 - **Interfaz de Inferencia:** Se diseñó una aplicación en Streamlit (`app.py`) en modo oscuro. Ofrece selectores para variables categóricas (Equipo, Rol, Mapa, Tipo de partida y el resultado `match_outcome`) y sliders para las numéricas.
 - **Inferencia Instantánea:** Al presionar "Predecir MVP", el sistema calcula la predicción en tiempo real y renderiza tarjetas personalizadas: un banner verde brillante con **JUGADOR MVP (Yes)** o uno estándar con **JUGADOR ESTÁNDAR (No)**.
-

4. Declaración de Uso de IA Generativa

De acuerdo con el compromiso de honestidad académica y las políticas de Duoc UC:

- **Uso del Asistente como Andamio de Aprendizaje:** Se declara el uso de IA generativa como un andamio cognitivo y técnico para el desarrollo de esta entrega. Las decisiones críticas del flujo experimental —como el diagnóstico del desbalanceo masivo en `match_outcome` y la re-orientación del target hacia `mvp_award`, junto con la arquitectura de los pipelines de preprocesamiento aislado para mitigar el data leakage— fueron debatidas, guiadas y estructuradas de forma conjunta con el asistente.
- **Soporte Técnico:** Adicionalmente, se utilizó la herramienta para el formateo del markdown del informe, la optimización del código del gráfico doble de subplots (curvas ROC/PR) y el diseño estético de la interfaz Streamlit en Python.